

## EXPERIMENT:6 FUNCTIONS

Q1.Develop a recursive and non recursive function FACT(num) to find the factorial of a number,  $n!$  defined by  $FACT(n)=1, n=0$ . Otherwise,  $FACT(n)=n*FACT(n-1)$ .using this function write a C program to compute the binomial coffecient . Tabulate the result for different values of  $n$  and  $r$  with suitable messages.

Algorithm:

Step 1: Start Step 2: Read values of  $n$  and  $r$  Step 3: If  $(r > n \text{ OR } n < 0 \text{ OR } r < 0)$

Print "Invalid Input" and Stop

Step 4: Compute factorial of  $n$  using recursive FACT( $n$ )

FACT\_rec( $n$ ): If  $n = 0 \rightarrow$  return 1 Else  $\rightarrow$  return  $n \times FACT\_rec(n-1)$

Step 5: Compute factorial of  $r$  using non-recursive FACT( $r$ )

FACT\_nonrec( $r$ ): Initialize  $fact = 1$  For  $i = 1$  to  $r$ :  $fact = fact \times i$  Return  $fact$

Step 6: Compute factorial of  $(n - r)$  using non-recursive FACT( $n-r$ ) Step 7: Compute Binomial Coefficient

$C(n, r) = FACT(n) / (FACT(r) \times FACT(n - r))$

Step 8: Display the value of  $C(n, r)$  Step 9: Print table of binomial coefficients

For  $i = 0$  to  $n$  For  $j = 0$  to  $i$  Compute  $C(i, j)$  Display result

Step 10: Stop

code:

```
#include <stdio.h>

// Recursive factorial function
long long FACT_rec(int n) {
    if (n == 0)
        return 1;
    else
        return n * FACT_rec(n - 1);
}

// Non-recursive factorial function
long long FACT_nonrec(int n) {
    long long fact = 1;
    for (int i = 1; i <= n; i++)
        fact *= i;
    return fact;
}

// Function to compute binomial coefficient C(n, r)
```

```

long long binomial(int n, int r) {
    long long num = FACT_rec(n);          // Using recursive factorial
    long long den = FACT_nonrec(r) * FACT_nonrec(n - r); // Using non-
recursive factorial
    return num / den;
}

int main() {
    int n, r;

    printf("BINOMIAL COEFFICIENT CALCULATOR USING FACTORIAL FUNCTIONS\n");
    printf("-----\n");

    printf("Enter value of n: ");
    scanf("%d", &n);

    printf("Enter value of r: ");
    scanf("%d", &r);

    if (r > n || n < 0 || r < 0) {
        printf("Invalid input! r must be <= n and both must be non-
negative.\n");
        return 1;
    }

    printf("\nUsing recursive FACT(n) and non-recursive FACT(r) & FACT(n-
r)...\n");
    printf("C(%d, %d) = %lld\n\n", n, r, binomial(n, r));

    // Display table
    printf("-----\n");
    printf("    TABLE OF BINOMIAL COEFFICIENTS (C(n, r))\n");
    printf("-----\n");
    printf("    n    r    C(n,r)\n");
    printf("-----\n");

    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= i; j++) {
            printf("    %3d %3d    %10lld\n", i, j, binomial(i, j));
        }
    }

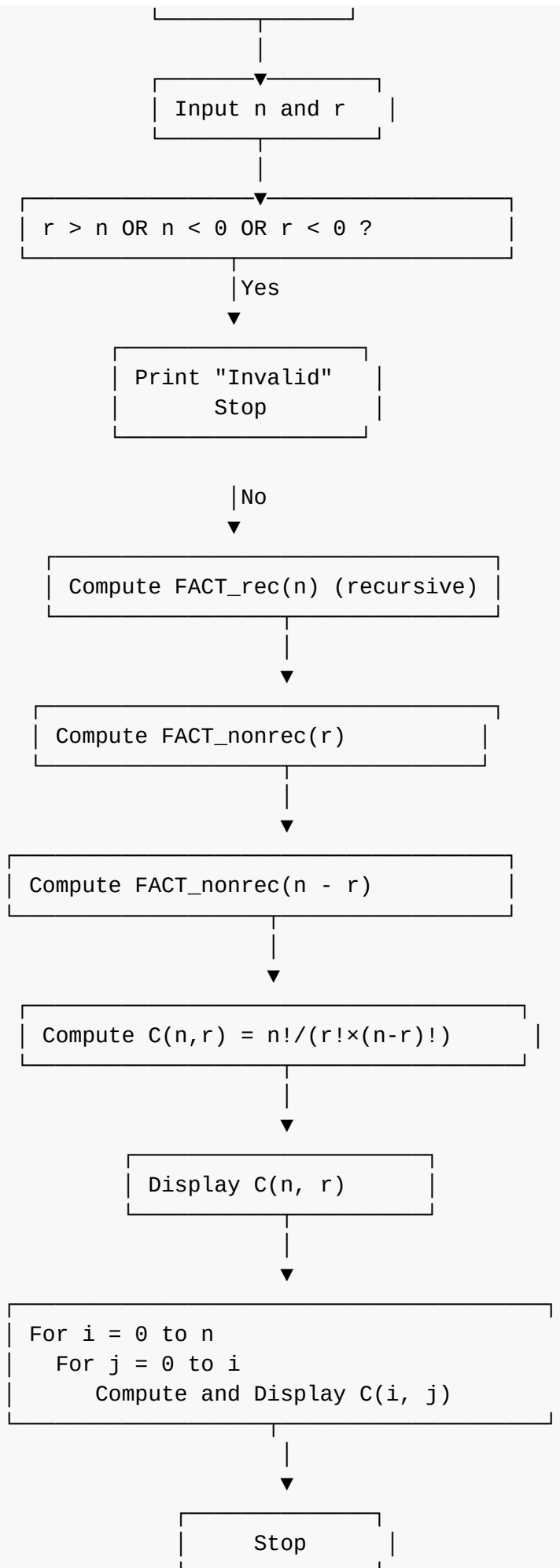
    printf("-----\n");

    return 0;
}

```

Flowchart:





## Output:

```

103 103
104 104
105 105
106 106
107 107
108 108
109 109
110 110
111 111
112 112
113 113
114 114
115 115
116 116
117 117
118 118
119 119
120 120
121 121
122 122
123 123
124 124
125 125
126 126
127 127
128 128
129 129
130 130
131 131
132 132
133 133
134 134
135 135
136 136
137 137
138 138
139 139
140 140
141 141
142 142
143 143
144 144
145 145
146 146
147 147
148 148
149 149
150 150
151 151
152 152
153 153
154 154
155 155
156 156
157 157
158 158
159 159
160 160
161 161
162 162
163 163
164 164
165 165
166 166
167 167
168 168
169 169
170 170
171 171
172 172
173 173
174 174
175 175
176 176
177 177
178 178
179 179
180 180
181 181
182 182
183 183
184 184
185 185
186 186
187 187
188 188
189 189
190 190
191 191
192 192
193 193
194 194
195 195
196 196
197 197
198 198
199 199
200 200
201 201
202 202
203 203
204 204
205 205
206 206
207 207
208 208
209 209
210 210
211 211
212 212
213 213
214 214
215 215
216 216
217 217
218 218
219 219
220 220
221 221
222 222
223 223
224 224
225 225
226 226
227 227
228 228
229 229
230 230
231 231
232 232
233 233
234 234
235 235
236 236
237 237
238 238
239 239
240 240
241 241
242 242
243 243
244 244
245 245
246 246
247 247
248 248
249 249
250 250
251 251
252 252
253 253
254 254
255 255
256 256
257 257
258 258
259 259
260 260
261 261
262 262
263 263
264 264
265 265
266 266
267 267
268 268
269 269
270 270
271 271
272 272
273 273
274 274
275 275
276 276
277 277
278 278
279 279
280 280
281 281
282 282
283 283
284 284
285 285
286 286
287 287
288 288
289 289
290 290
291 291
292 292
293 293
294 294
295 295
296 296
297 297
298 298
299 299
300 300
301 301
302 302
303 303
304 304
305 305
306 306
307 307
308 308
309 309
310 310
311 311
312 312
313 313
314 314
315 315
316 316
317 317
318 318
319 319
320 320
321 321
322 322
323 323
324 324
325 325
326 326
327 327
328 328
329 329
330 330
331 331
332 332
333 333
334 334
335 335
336 336
337 337
338 338
339 339
340 340
341 341
342 342
343 343
344 344
345 345
346 346
347 347
348 348
349 349
350 350
351 351
352 352
353 353
354 354
355 355
356 356
357 357
358 358
359 359
360 360
361 361
362 362
363 363
364 364
365 365
366 366
367 367
368 368
369 369
370 370
371 371
372 372
373 373
374 374
375 375
376 376
377 377
378 378
379 379
380 380
381 381
382 382
383 383
384 384
385 385
386 386
387 387
388 388
389 389
390 390
391 391
392 392
393 393
394 394
395 395
396 396
397 397
398 398
399 399
400 400
401 401
402 402
403 403
404 404
405 405
406 406
407 407
408 408
409 409
410 410
411 411
412 412
413 413
414 414
415 415
416 416
417 417
418 418
419 419
420 420
421 421
422 422
423 423
424 424
425 425
426 426
427 427
428 428
429 429
430 430
431 431
432 432
433 433
434 434
435 435
436 436
437 437
438 438
439 439
440 440
441 441
442 442
443 443
444 444
445 445
446 446
447 447
448 448
449 449
450 450
451 451
452 452
453 453
454 454
455 455
456 456
457 457
458 458
459 459
460 460
461 461
462 462
463 463
464 464
465 465
466 466
467 467
468 468
469 469
470 470
471 471
472 472
473 473
474 474
475 475
476 476
477 477
478 478
479 479
480 480
481 481
482 482
483 483
484 484
485 485
486 486
487 487
488 488
489 489
490 490
491 491
492 492
493 493
494 494
495 495
496 496
497 497
498 498
499 499
500 500
501 501
502 502
503 503
504 504
505 505
506 506
507 507
508 508
509 509
510 510
511 511
512 512
513 513
514 514
515 515
516 516
517 517
518 518
519 519
520 520
521 521
522 522
523 523
524 524
525 525
526 526
527 527
528 528
529 529
530 530
531 531
532 532
533 533
534 534
535 535
536 536
537 537
538 538
539 539
540 540
541 541
542 542
543 543
544 544
545 545
546 546
547 547
548 548
549 549
550 550
551 551
552 552
553 553
554 554
555 555
556 556
557 557
558 558
559 559
560 560
561 561
562 562
563 563
564 564
565 565
566 566
567 567
568 568
569 569
570 570
571 571
572 572
573 573
574 574
575 575
576 576
577 577
578 578
579 579
580 580
581 581
582 582
583 583
584 584
585 585
586 586
587 587
588 588
589 589
590 590
591 591
592 592
593 593
594 594
595 595
596 596
597 597
598 598
599 599
600 600
601 601
602 602
603 603
604 604
605 605
606 606
607 607
608 608
609 609
610 610
611 611
612 612
613 613
614 614
615 615
616 616
617 617
618 618
619 619
620 620
621 621
622 622
623 623
624 624
625 625
626 626
627 627
628 628
629 629
630 630
631 631
632 632
633 633
634 634
635 635
636 636
637 637
638 638
639 639
640 640
641 641
642 642
643 643
644 644
645 645
646 646
647 647
648 648
649 649
650 650
651 651
652 652
653 653
654 654
655 655
656 656
657 657
658 658
659 659
660 660
661 661
662 662
663 663
664 664
665 665
666 666
667 667
668 668
669 669
670 670
671 671
672 672
673 673
674 674
675 675
676 676
677 677
678 678
679 679
680 680
681 681
682 682
683 683
684 684
685 685
686 686
687 687
688 688
689 689
690 690
691 691
692 692
693 693
694 694
695 695
696 696
697 697
698 698
699 699
700 700
701 701
702 702
703 703
704 704
705 705
706 706
707 707
708 708
709 709
710 710
711 711
712 712
713 713
714 714
715 715
716 716
717 717
718 718
719 719
720 720
721 721
722 722
723 723
724 724
725 725
726 726
727 727
728 728
729 729
730 730
731 731
732 732
733 733
734 734
735 735
736 736
737 737
738 738
739 739
740 740
741 741
742 742
743 743
744 744
745 745
746 746
747 747
748 748
749 749
750 750
751 751
752 752
753 753
754 754
755 755
756 756
757 757
758 758
759 759
760 760
761 761
762 762
763 763
764 764
765 765
766 766
767 767
768 768
769 769
770 770
771 771
772 772
773 773
774 774
775 775
776 776
777 777
778 778
779 779
780 780
781 781
782 782
783 783
784 784
785 785
786 786
787 787
788 788
789 789
790 790
791 791
792 792
793 793
794 794
795 795
796 796
797 797
798 798
799 799
800 800
801 801
802 802
803 803
804 804
805 805
806 806
807 807
808 808
809 809
810 810
811 811
812 812
813 813
814 814
815 815
816 816
817 817
818 818
819 819
820 820
821 821
822 822
823 823
824 824
825 825
826 826
827 827
828 828
829 829
830 830
831 831
832 832
833 833
834 834
835 835
836 836
837 837
838 838
839 839
840 840
841 841
842 842
843 843
844 844
845 845
846 846
847 847
848 848
849 849
850 850
851 851
852 852
853 853
854 854
855 855
856 856
857 857
858 858
859 859
860 860
861 861
862 862
863 863
864 864
865 865
866 866
867 867
868 868
869 869
870 870
871 871
872 872
873 873
874 874
875 875
876 876
877 877
878 878
879 879
880 880
881 881
882 882
883 883
884 884
885 885
886 886
887 887
888 888
889 889
890 890
891 891
892 892
893 893
894 894
895 895
896 896
897 897
898 898
899 899
900 900
901 901
902 902
903 903
904 904
905 905
906 906
907 907
908 908
909 909
910 910
911 911
912 912
913 913
914 914
915 915
916 916
917 917
918 918
919 919
920 920
921 921
922 922
923 923
924 924
925 925
926 926
927 927
928 928
929 929
930 930
931 931
932 932
933 933
934 934
935 935
936 936
937 937
938 938
939 939
940 940
941 941
942 942
943 943
944 944
945 945
946 946
947 947
948 948
949 949
950 950
951 951
952 952
953 953
954 954
955 955
956 956
957 957
958 958
959 959
960 960
961 961
962 962
963 963
964 964
965 965
966 966
967 967
968 968
969 969
970 970
971 971
972 972
973 973
974 974
975 975
976 976
977 977
978 978
979 979
980 980
981 981
982 982
983 983
984 984
985 985
986 986
987 987
988 988
989 989
990 990
991 991
992 992
993 993
994 994
995 995
996 996
997 997
998 998
999 999
1000 1000

```

Q2. Develop a recursive GCD function(num1,num2) that accepts two integer arguments .write a c program that invokes this function to find greatest common divisor of two given integers.

## Algorithm:

Step 1: Start Step 2: Read two integers num1 and num2 Step 3: Call the function GCD(num1, num2) Inside GCD(a, b):

If b == 0, return a

Otherwise, return GCD(b, a % b)

Step 4: Display the returned GCD value Step 5: Stop

## Code:

```

#include <stdio.h>

// Recursive function to find GCD
int GCD(int num1, int num2) {
    if (num2 == 0)
        return num1;
    else
        return GCD(num2, num1 % num2);
}

int main() {

```

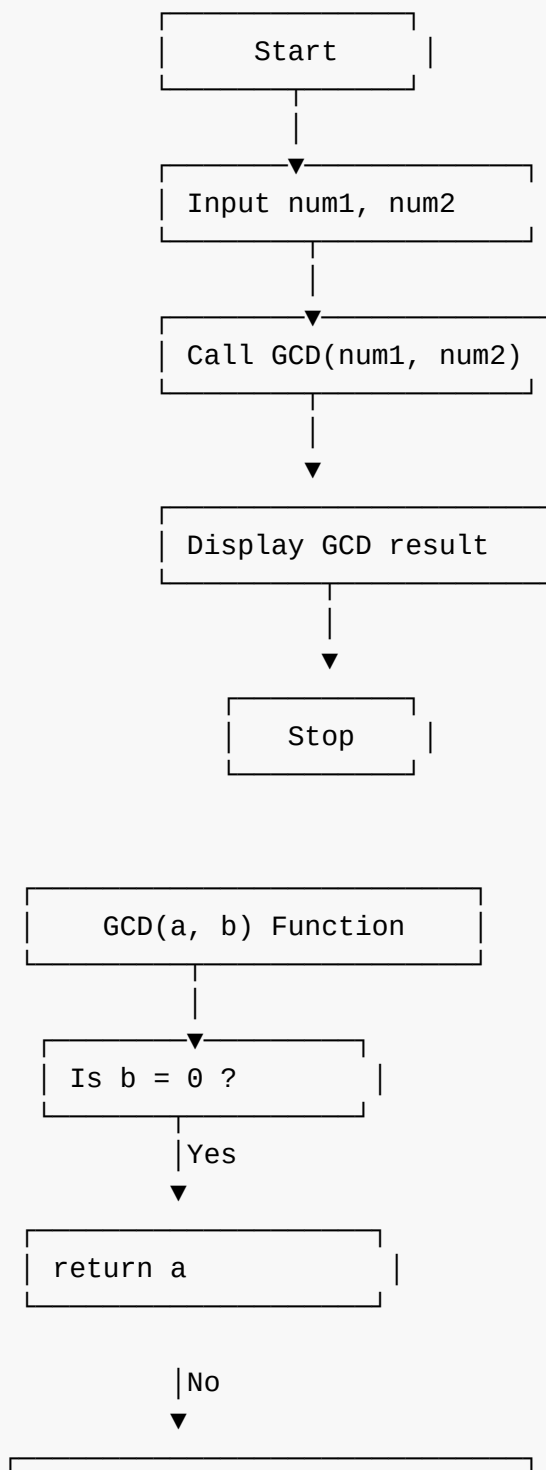
```
int a, b;

printf("Enter two integers: ");
scanf("%d %d", &a, &b);

printf("The GCD of %d and %d is: %d\n", a, b, GCD(a, b));

return 0;
}
```

### Flowchart:



```
return GCD(b, a % b)
```

## Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q2.c -o exp6Q2 && "/home/aaditya/experiments-in-c/exp6/"exp6Q2
• aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q2.c -o exp6Q2 && "/home/aaditya/experiments-in-c/exp6/"exp6Q2
Enter two integers: 1 2
The GCD of 1 and 2 is: 1
○ aaditya@fedora:~/experiments-in-c/exp6$
```

Q3. Develop a recursive function FIBO(num) that accepts an integer argument. Write a C program that invokes this function to generate the Fibonacci sequence up to num.

## Algorithm:

Step 1: Start Step 2: Read the value of num (the limit of the sequence) Step 3: For each value i from 0 to num-1

Call FIBO(i)      Display the returned value

Step 4: End loop Step 5: Stop Recursive Function FIBO(n):

If n == 0, return 0

If n == 1, return 1

Otherwise return FIBO(n-1) + FIBO(n-2)

## Code:

```
#include <stdio.h>

// Recursive Fibonacci function
int FIBO(int num) {
    if (num == 0)
        return 0;
    if (num == 1)
        return 1;
    return FIBO(num - 1) + FIBO(num - 2);
}

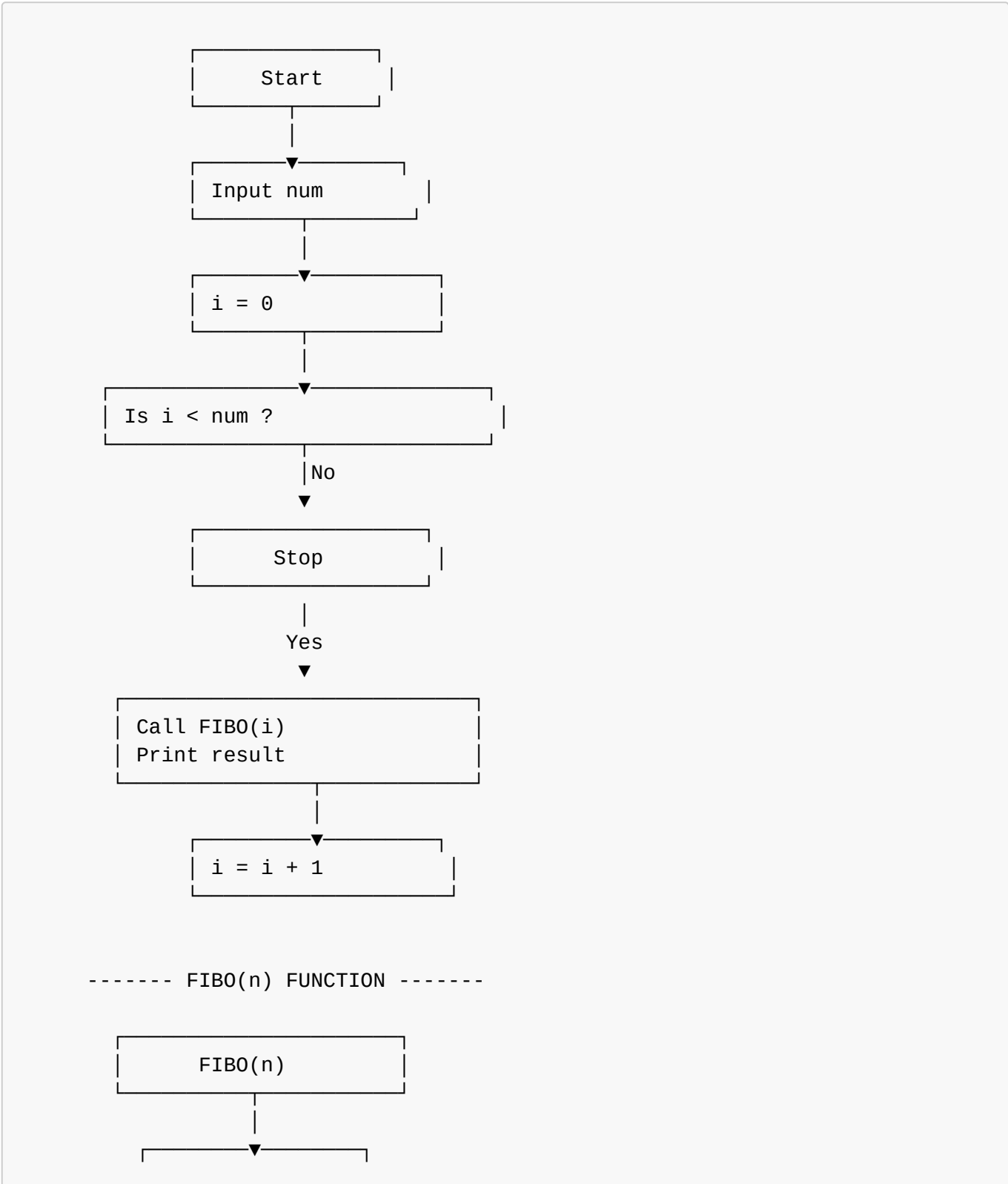
int main() {
    int num;

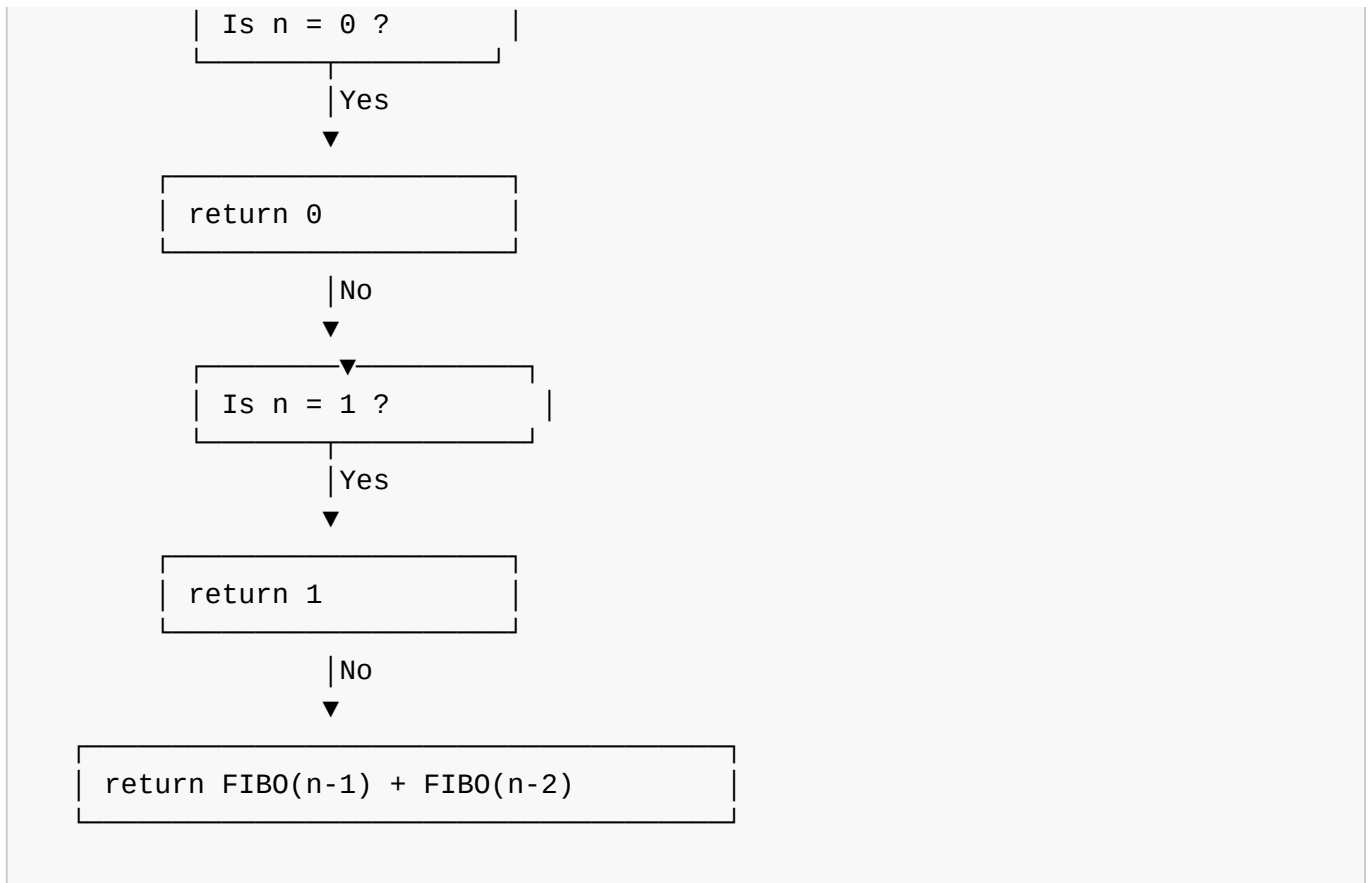
    printf("Enter the number of terms: ");
    scanf("%d", &num);

    printf("Fibonacci Sequence up to %d terms:\n", num);
```

```
for (int i = 0; i < num; i++) {  
    printf("%d ", FIB0(i));  
}  
  
printf("\n");  
  
return 0;  
}
```

Flowchart:





Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q3.c -o exp6Q3 && "/home/aaditya/experiments-in-c/exp6/"exp6Q3
aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q3.c -o exp6Q3 && "/home/aaditya/experiments-in-c/exp6/"exp6Q3
Enter the number of terms: 2
Fibonacci Sequence up to 2 terms:
0 1
aaditya@fedora:~/experiments-in-c/exp6$

```

Q4. Develop a C function ISPRIME (num) that accepts an integer argument and returns 1 if the argument is prime, a 0 otherwise. Write a C program that invokes this function to generate prime numbers between the given ranges.

Algorithm:

Step 1: Start Step 2: Read the lower limit start and upper limit end Step 3: For each number i from start to end

Call ISPRIME(i) If function returns 1 → Print i

Step 4: End loop Step 5: Stop Algorithm for ISPRIME(num):

If num ≤ 1, return 0

For i = 2 to num/2 If num % i == 0, return 0

Return 1 (number is prime)

Code:



```
#include <stdio.h>

// Function to check if number is prime
int ISPRIME(int num) {
    if (num <= 1)
        return 0;

    for (int i = 2; i <= num / 2; i++) {
        if (num % i == 0)
            return 0; // Not prime
    }

    return 1; // Prime
}

int main() {
    int start, end;

    printf("Enter the start of range: ");
    scanf("%d", &start);

    printf("Enter the end of range: ");
    scanf("%d", &end);

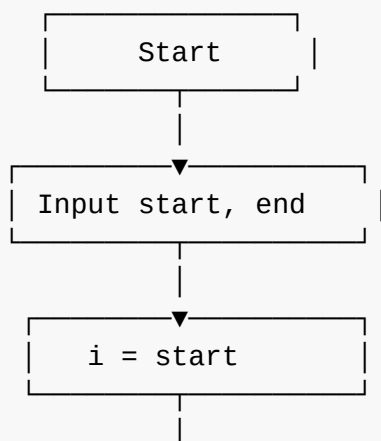
    printf("Prime numbers between %d and %d are:\n", start, end);

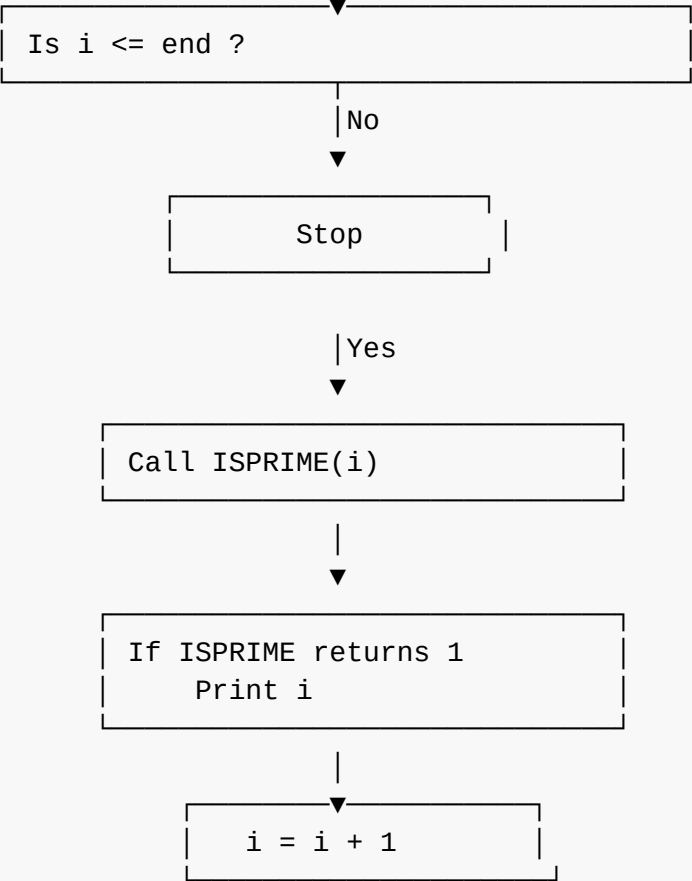
    for (int i = start; i <= end; i++) {
        if (ISPRIME(i))
            printf("%d ", i);
    }

    printf("\n");

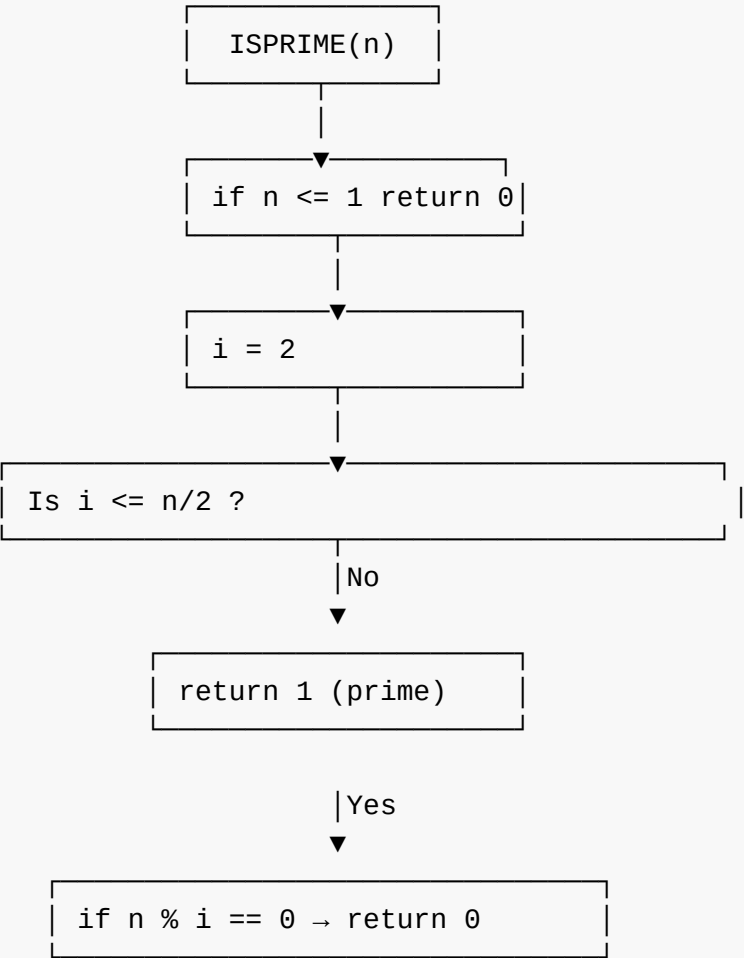
    return 0;
}
```

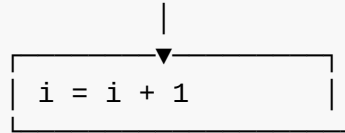
Flowchart:





----- FLOWCHART FOR ISPRIME(num) -----





Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q4.c -o exp6Q4 && "/home/aaditya/experiments-in-c/exp6/"exp6Q4
aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q4.c -o exp6Q4 && "/home/aaditya/experiments-in-c/exp6/"exp6Q4
Enter the start of range: 4
Enter the end of range: 2
Prime numbers between 4 and 2 are:

aaditya@fedora:~/experiments-in-c/exp6$ cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q4.c -o exp6Q4 && "/home/aaditya/experiments-in-c/exp6/"exp6Q4
Enter the start of range: 1
Enter the end of range: 10
Prime numbers between 1 and 10 are:
2 3 5 7
aaditya@fedora:~/experiments-in-c/exp6$
  
```

Q5. Develop a function REVERSE (str) that accepts a string argument. Write a C program that invokes this function to find the reverse of a given string.

Algorithm:

Step 1: Start Step 2: Input a string str Step 3: Call the function REVERSE(str) Step 4: Inside REVERSE(str):

Find the length of the string

Swap characters from start and end

Continue until the middle of the string

Step 5: Return the reversed string Step 6: Display the reversed string in main() Step 7: Stop

Code:

```

#include <stdio.h>
#include <string.h>

// Function to reverse a string
void REVERSE(char str[]) {
    int i, j;
    char temp;

    j = strlen(str) - 1; // last index

    for (i = 0; i < j; i++, j--) {
        temp = str[i];
        str[i] = str[j];
        str[j] = temp;
    }
}

int main() {
  
```

```

char str[100];

printf("Enter a string: ");
gets(str);  // (Use fgets in modern compilers)

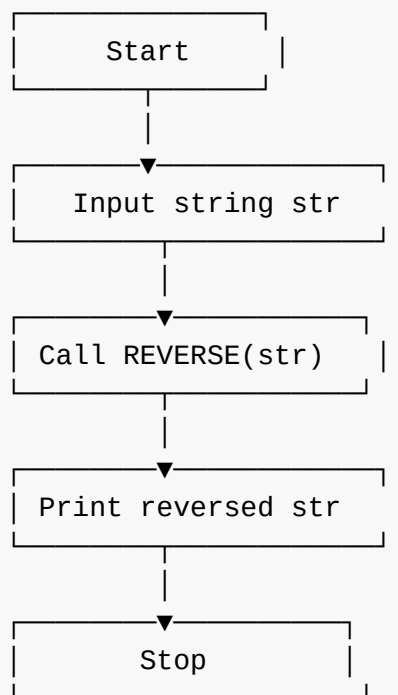
REVERSE(str);  // function call

printf("Reversed string: %s\n", str);

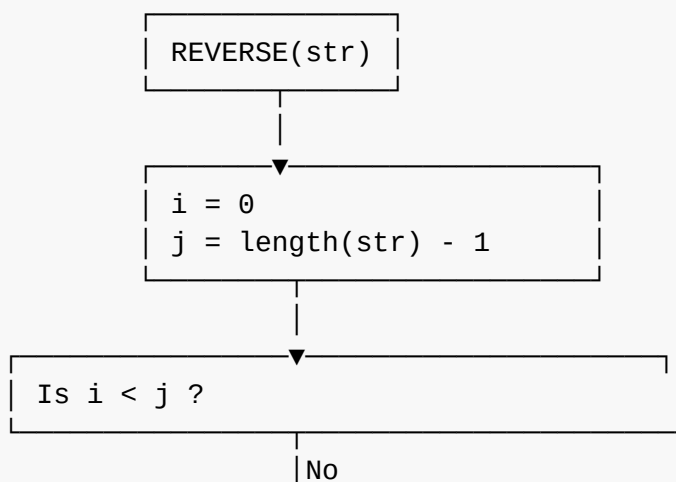
return 0;
}

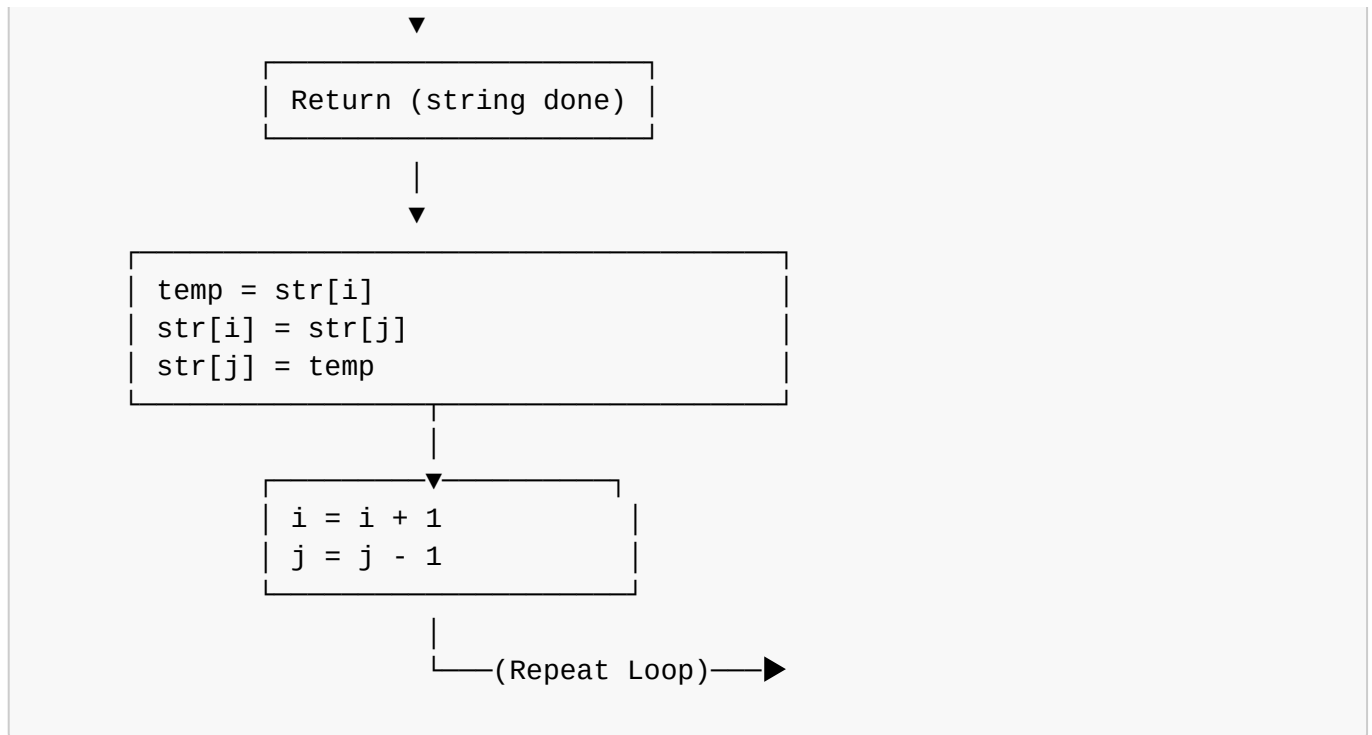
```

### Flowchart:



----- FUNCTION: REVERSE(str) -----





Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q5.c -o exp6Q5 && "/home/aaditya/experiments-in-c/exp6/"exp6Q5
• aaditya@fedora:~/experiments-in-c$ cd "/home/aaditya/experiments-in-c/exp6/" && gcc exp6Q5.c -o exp6Q5 && "/home/aaditya/experiments-in-c/exp6/"exp6Q5
Enter a string: aditya
Reversed string: ytidA
◦ aaditya@fedora:~/experiments-in-c/exp6$
```